

Build the Python Script (app.py)

Description

Setup and Initialization

- **Import Dependencies:** Load the Flask module and joblib.
- **Initialize App:** Create the application instance using Flask(**name**).
- **Load Assets:** Load the saved model ([floods.save](#)) and scaler ([transform.save](#)) to process inputs and generate predictions.

Page Routing: URL endpoints are bound to HTML templates using the @app.route() decorator and render_template():

- / routes to home.html (Landing page).
- /Predict routes to index.html (Input form).
- /chance routes to chance.html (Flood predicted).
- /no_chance routes to no_chance.html (No flood predicted).

Prediction Logic

- The predict() function receives weather parameters via a POST request.
- The input values are structured into a DataFrame containing the five independent features.
- The data is passed to the loaded model to generate a prediction, which then redirects the user to the appropriate result page.

Execution

- The application is executed from the main block using [app.run\(\)](#), launching the development server on localhost.

app.py implementation for a Flask flood prediction application. It assumes your templates (home.html, index.html, chance.html, and no_chance.html) already exist and that your saved model files are named floods.save and transform.save.

```
from flask import Flask, render_template, request, redirect, url_for
```

```
import joblib
```

```
import pandas as pd
```

```
# Initialize Flask application
```

```
app = Flask(__name__)
```

```
# Load trained model and scaler
```

```
model = joblib.load("floods.save")
```

```
scaler = joblib.load("transform.save")
```

```
# Home Page

@app.route("/")
def home():
    return render_template("home.html")


# Prediction Form Page

@app.route("/Predict")
def Predict():
    return render_template("index.html")


# Flood Prediction Result Page

@app.route("/chance")
def chance():
    return render_template("chance.html")


# No Flood Prediction Result Page

@app.route("/no_chance")
def no_chance():
    return render_template("no_chance.html")


# Prediction Logic

@app.route("/predict", methods=["POST"])
def predict():
    try:
        # Read input values from form
        feature1 = float(request.form["feature1"])
        feature2 = float(request.form["feature2"])
        feature3 = float(request.form["feature3"])
        feature4 = float(request.form["feature4"])
        feature5 = float(request.form["feature5"])

        # Create DataFrame
        input_data = pd.DataFrame
```

```
[[feature1, feature2, feature3, feature4, feature5]],
```

```
    columns=[
```

```
        "feature1",
```

```
        "feature2",
```

```
        "feature3",
```

```
        "feature4",
```

```
        "feature5"
```

```
    ]
```

```
)
```

```
# Scale the input
```

```
scaled_data = scaler.transform(input_data)
```

```
# Predict
```

```
prediction = model.predict(scaled_data)
```

```
# Redirect based on prediction
```

```
if prediction[0] == 1:
```

```
    return redirect(url_for("chance"))
```

```
else:
```

```
    return redirect(url_for("no_chance"))
```

```
except Exception as e:
```

```
    return f"Error: {e}"
```

```
# Run the application
```

```
if __name__ == "__main__":
```

```
    app.run(debug=True)
```

```
from flask import Flask, render_template, request
# used to run/serve our application
# render_template is used for rendering the html pages
# import load from joblib to load the saved model file
from joblib import load
```

Create Flask app and Load our model file.

```
app=Flask(__name__) # our flask app
#load model file
model =load('floods.save')
sc=load('transform.save')
```